

Nome do Aluno:	RA:
-----------------------	------------

Curso: Ciência da Computação	
Disciplina: Paradigma de Linguagens de Programação	Campus: Passo Fundo
Professor (a): Luciano Rodrigo Ferretto	Data: ____/____/2026

CARTÃO DE RESPOSTAS

Assinatura do Aluno	ATENÇÃO! Instruções para preenchimento 1. Não dobre ou rasure este cartão de respostas 2. Marque apenas uma resposta por questão 3. Todas as questões devem ser respondidas com caneta azul ou preta 4. Nas dissertativas, caso o espaço não seja suficiente, pode-se usar o verso da folha. 5. Preencher totalmente o espaço conforme o modelo abaixo: ☒ Forma correta de preenchimento
----------------------------	--

Questões Objetivas

01	02	03	04	05	06	07	08	09	10	11	12	13	14	15
A	A	A	A	A	A	A	A	A	A	A	A	A	A	A
B	B	B	B	B	B	B	B	B	B	B	B	B	B	B
C	C	C	C	C	C	C	C	C	C	C	C	C	C	C
D	D	D	D	D	D	D	D	D	D	D	D	D	D	D
E	E	E	E	E	E	E	E	E	E	E	E	E	E	E

Questões Dissertativas nas próximas folhas

Orientações:

- A prova teórica é individual e sem consulta ao material.
- **Lembre-se de colocar as respostas das questões objetivas no cartão da primeira página, e nas páginas seguintes as respostas das questões dissertativas.**
 - **NÃO SERÁ CORRIGIDO NADA ALÉM DESSAS 2 (DUAS) PRIMEIRAS PÁGINAS.**
- Questões Objetivas: 6,2
- Questões Dissertativas: 0,8

UMA ÓTIMA PROVA A TODOS!



Questão Dissertativa 16:

A large, faint, light gray letter 'A' is centered on a background of horizontal ruling lines. The 'A' is composed of two slanted lines meeting at a point at the top, with a horizontal crossbar. The ruling lines are thin and dark gray, spaced evenly across the page. The 'A' is positioned such that its top is near the top of the page and its base is near the bottom, spanning most of the page's width.

1) Durante uma aula prática, um estudante afirma que programação procedural e programação orientada a objetos pertencem a categorias completamente independentes, pois uma utiliza funções e a outra utiliza objetos. Considerando a classificação dos paradigmas estudados, analise as afirmações.

- I.** A programação procedural está associada aos paradigmas imperativos.
- II.** A programação orientada a objetos também pode ser classificada como uma abordagem imperativa.
- III.** Ambas podem utilizar mecanismos que representam ações realizadas durante a execução do programa.
- IV.** A programação orientada a objetos pertence necessariamente ao grupo dos paradigmas lógicos.

Assinale a alternativa correta.

- A)** Apenas I e II estão corretas.
- B)** Apenas I, II e III estão corretas.
- C)** Apenas I, III e IV estão corretas.
- D)** Apenas II, III e IV estão corretas.
- E)** I, II, III e IV estão corretas.

2) Um desenvolvedor cria uma classe ContaBancaria que mantém o saldo como um atributo interno. O saldo não pode ser alterado diretamente por outras classes; operações como depósito e saque são realizadas por métodos que controlam as regras necessárias.

Considerando os pilares da Programação Orientada a Objetos, essa característica representa principalmente:

- A)** abstração, pois elimina completamente os detalhes do objeto.
- B)** herança, pois concentra o comportamento em uma classe especializada.
- C)** encapsulamento, pois restringe o acesso direto ao estado interno.
- D)** polimorfismo, pois permite diferentes implementações do mesmo método.
- E)** composição, pois permite que objetos compartilhem seus atributos internos.

3) Um programa funcional recebe uma coleção de produtos e utiliza uma função para transformar cada elemento em seu respectivo preço. Em seguida, outra operação seleciona somente os preços que atendem a determinada condição.

A abordagem descrita representa principalmente:

- A)** transformação e composição de funções sobre uma coleção de dados.
- B)** criação de objetos especializados por meio de mecanismos de herança.
- C)** utilização de fatos e regras para realizar inferências sobre os produtos.
- D)** alteração sequencial de variáveis para controlar o estado da aplicação.
- E)** encapsulamento dos dados da coleção em uma hierarquia de classes.

4) Em um sistema baseado no paradigma lógico, foram definidos os seguintes elementos conceituais:

- fatos sobre determinados objetos do domínio;
- regras que estabelecem relações entre esses fatos;
- consultas realizadas pelo usuário.

A partir desses elementos, o sistema consegue obter respostas que não foram necessariamente cadastradas como fatos explícitos.

Nesse contexto, o processo responsável por obter essas respostas está relacionado:

- A)** à execução sequencial de procedimentos definidos pelo programador.
- B)** à inferência realizada a partir dos fatos e das regras existentes.
- C)** à transformação dos dados por meio de funções puras e imutáveis.
- D)** à criação de objetos que encapsulam os fatos e as regras do domínio.
- E)** à utilização de estruturas de repetição para testar todas as possibilidades.

5) Um programador possui duas funções:

- $f(x)$, que sempre produz o mesmo resultado para determinado x ;
- $g(x)$, que consulta uma variável global antes de produzir o resultado.

Considerando o conceito de função pura, analise as afirmações.

- I.** $f(x)$ apresenta uma característica esperada de uma função pura.
- II.** O acesso ao estado externo pode introduzir efeitos que dificultam a previsibilidade de $g(x)$.
- III.** Funções puras favorecem testes porque seus resultados dependem das entradas fornecidas.

∴

IV. Uma função pura deve necessariamente modificar alguma variável externa para produzir seu resultado.

Assinale a alternativa correta.

- A)** Apenas I e II estão corretas.
- B)** Apenas I, II e III estão corretas.
- C)** Apenas I, III e IV estão corretas.
- D)** Apenas II, III e IV estão corretas.
- E)** I, II, III e IV estão corretas.

6) Durante uma revisão de código, foram encontradas quatro situações:

- um trecho de código de validação foi copiado para cinco classes;
- uma funcionalidade futura foi implementada sem fazer parte dos requisitos;
- uma classe contém métodos sem relação entre si;
- uma alteração em uma classe frequentemente exige mudanças em várias outras.

Considerando os conceitos estudados, analise as afirmações.

I. A duplicação da validação está relacionada ao princípio DRY.

II. A implementação antecipada de uma funcionalidade pode caracterizar violação de YAGNI.

III. A reunião de responsabilidades pouco relacionadas pode indicar baixa coesão.

IV. A necessidade de alterações em várias classes pode indicar alto acoplamento.

Assinale a alternativa correta.

- A)** Apenas I e II estão corretas.
- B)** Apenas I, II e III estão corretas.
- C)** Apenas I, III e IV estão corretas.
- D)** Apenas II, III e IV estão corretas.
- E)** I, II, III e IV estão corretas.

7) Uma equipe possui uma classe que realiza operações de pagamento, gera relatórios financeiros e envia notificações aos clientes. Um dos desenvolvedores propõe dividir essas responsabilidades em classes especializadas.

Considerando os princípios de design estudados, a proposta está principalmente relacionada à busca por:

- A)** menor coesão, permitindo que cada classe tenha várias responsabilidades.
- B)** maior acoplamento, concentrando as dependências entre os componentes.
- C)** maior coesão, mantendo responsabilidades relacionadas em cada componente.
- D)** maior duplicação, evitando que responsabilidades sejam compartilhadas entre classes.
- E)** maior complexidade, distribuindo as mesmas responsabilidades entre vários componentes.

8) Uma aplicação possui uma interface Funcionario com métodos relacionados a diferentes atividades. Algumas classes que implementam essa interface utilizam apenas parte dos métodos, enquanto outras precisam fornecer implementações vazias para métodos que não fazem sentido em seu contexto.

Considerando os princípios SOLID, essa situação está mais diretamente relacionada ao:

- A)** SRP, pois a interface contém apenas uma responsabilidade.
- B)** OCP, pois a interface permite que novas classes sejam adicionadas.
- C)** LSP, pois as implementações possuem comportamentos diferentes.
- D)** ISP, pois clientes são obrigados a depender de métodos que não utilizam.
- E)** DIP, pois a interface representa uma dependência de baixo nível.

9) Uma classe Pedido possui diretamente uma instância de RepositorioMySQL. Em uma nova implementação, a equipe cria uma interface Repositorio e faz Pedido depender dessa abstração, permitindo fornecer diferentes implementações de armazenamento.

Considerando os princípios SOLID, a alteração realizada está relacionada principalmente ao:

- A)** SRP, pois elimina todas as responsabilidades da classe Pedido.
- B)** OCP, pois impede modificações futuras na classe Pedido.
- C)** LSP, pois substitui uma classe-base por uma classe derivada.
- D)** ISP, pois transforma todos os métodos em interfaces independentes.
- E)** DIP, pois reduz a dependência direta de uma implementação concreta.



10) Uma classe Relatorio possui um método responsável por gerar diferentes tipos de relatórios. Para cada novo tipo, o desenvolvedor precisa modificar a classe e acrescentar uma nova condição.

A equipe pretende reorganizar o projeto para permitir que novos tipos de relatório sejam adicionados sem modificar continuamente a classe principal.

Essa mudança está diretamente relacionada ao princípio:

- A) SRP, por reduzir a quantidade de atributos da classe.
- B) OCP, por favorecer extensão sem alterações constantes no código existente.
- C) LSP, por permitir substituir qualquer relatório por outro.
- D) ISP, por eliminar interfaces com muitos métodos.
- E) DIP, por obrigar todas as classes a utilizar a mesma implementação.

11) Uma empresa possui uma aplicação monolítica em que todas as funcionalidades são implantadas como uma única unidade. Após uma alteração em apenas uma funcionalidade, toda a aplicação precisa ser reconstruída e implantada novamente.

Considerando as características estudadas, analise as afirmações.

- I. A situação representa uma característica comum de aplicações monolíticas.
- II. A divisão em microservices pode permitir que partes do sistema sejam implantadas independentemente.
- III. Microservices podem permitir escalabilidade individual dos serviços.
- IV. A adoção de microservices elimina todos os custos relacionados à implantação e operação.

Assinale a alternativa correta.

- A) Apenas I e II estão corretas.
- B) Apenas I, II e III estão corretas.
- C) Apenas I, III e IV estão corretas.
- D) Apenas II, III e IV estão corretas.
- E) I, II, III e IV estão corretas.

12) Durante a decomposição de um sistema, a equipe identifica que o serviço de pedidos necessita dos dados de produtos, mas não deve acessar diretamente as tabelas pertencentes ao serviço de produtos.

Considerando o **Database per Service Pattern**, qual alternativa representa melhor essa abordagem?

- A) O serviço de pedidos deve acessar diretamente o banco do serviço de produtos.
- B) Os dois serviços devem compartilhar todas as tabelas para evitar inconsistências.
- C) Cada serviço deve controlar seus próprios dados e utilizar comunicação entre serviços quando necessário.
- D) O banco do serviço de produtos deve disponibilizar todas as tabelas para leitura externa.
- E) O serviço de produtos deve transferir permanentemente seu banco para o serviço de pedidos.

13) Uma aplicação utiliza Flyway para controlar alterações no banco de dados. Após a aplicação das migrações V1 e V2, um desenvolvedor identifica que uma nova alteração é necessária.

Considerando a estratégia de migrações versionadas, qual alternativa representa a abordagem adequada?

- A) Modificar a V1, pois todas as alterações devem permanecer no primeiro script.
- B) Modificar a V2, pois somente a última migração pode ser alterada.
- C) Criar uma nova migração versionada contendo a alteração necessária.
- D) Excluir as migrações anteriores e substituí-las pela estrutura atual do banco.
- E) Alterar somente o código Java e deixar o banco ser atualizado automaticamente pela aplicação.

O material apresenta explicitamente a criação de uma nova migração para alterações posteriores às migrações já aplicadas.

14) Uma empresa possui um microservice que utiliza banco local e logs detalhados em desenvolvimento, banco de homologação e logs moderados em testes, e banco corporativo e logs restritos em produção.

A equipe deseja manter o mesmo código da aplicação e selecionar o conjunto adequado de configurações durante a execução.

Considerando **Configuration Profiles**, a solução mais adequada é:

- A) criar uma implementação diferente do microservice para cada ambiente.
- B) criar um perfil para cada ambiente e ativar o perfil correspondente.



- C) manter todas as configurações fixas no código para evitar diferenças entre ambientes.
- D) utilizar somente as configurações de produção em todos os ambientes.
- E) duplicar as classes Java e alterar seus valores conforme o ambiente.

15) Uma equipe utiliza um mecanismo de configuração centralizada para seus microservices. Em determinado momento, os desenvolvedores afirmam que essa centralização garante automaticamente que qualquer alteração feita na configuração será aplicada imediatamente a todos os serviços.

Considerando os conceitos estudados, essa afirmação é:

- A) correta, pois toda configuração centralizada possui atualização dinâmica obrigatória.
- B) correta, pois a centralização elimina a necessidade de reinicialização dos serviços.
- C) incorreta, pois a centralização organiza as configurações, enquanto recursos dinâmicos dependem da solução adotada.
- D) incorreta, pois configurações centralizadas somente podem ser utilizadas durante o desenvolvimento.
- E) correta, pois um servidor central sempre possui controle direto sobre a execução dos serviços.

16) Considere o código Java a seguir:

```

1  abstract class Veiculo {
2      public abstract void acelerar();
3  }
4  class Carro extends Veiculo {
5      @Override
6      public void acelerar() {
7          System.out.println("O carro está acelerando com o motor...");
8      }
9  }
10 class Moto extends Veiculo {
11     @Override
12     public void acelerar() {
13         System.out.println("A Moto está acelerando com o motor...");
14     }
15 }
16 class Bicicleta extends Veiculo {
17     @Override
18     public void acelerar() {
19         throw new UnsupportedOperationException("Bicicletas não possuem motor para acelerar!");
20     }
21 }
22 public class Main {
23     private static void acelerar(Veiculo veiculo) {
24         veiculo.acelerar();
25     }
26     public static void main(String[] args) {
27         Veiculo v1 = new Carro();
28         Veiculo v2 = new Moto();
29         Veiculo v3 = new Bicicleta();
30         acelerar(v1);
31         acelerar(v2);
32         acelerar(v3);
33     }
34 }

```

O método `acelerar()` é definido na classe abstrata `Veiculo` e, portanto, qualquer objeto recebido pelo método `acelerar(Veiculo veiculo)` na classe `Main` deve ser capaz de atender ao comportamento esperado dessa abstração. Entretanto, a implementação de `Bicicleta` não consegue realizar a operação definida por sua classe-base e lança uma exceção.

Com base nos princípios SOLID estudados na disciplina:

- a) Identifique o princípio SOLID violado pelo código e explique por que a implementação da classe `Bicicleta` não respeita esse princípio.
- b) Proponha uma alteração na hierarquia de classes que permita representar corretamente os diferentes tipos de veículos, evitando que uma classe herde um comportamento que não consegue implementar. Apresente o código ou a estrutura das classes proposta e explique por que sua solução respeita o princípio identificado.